Task 8 – Plan the Algorithm Before Coding

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task08)

1. Steps of algorithmic problem solving

Step	Applied to the milk payment
1. Understand	Input: the entries of one farmer for ten days. Output: amount payable. Rules: Task 1.
2. Plan	Pseudocode and flowchart below; hand trace on three entries before any Python is written.
3. Write	Translate the pseudocode line by line into functions (Tasks 6, 9).
4. Test	Compare program output with the hand trace and the five-entry hand total (Task 9).

2. Pseudocode – one payment and the ten-day total for one farmer

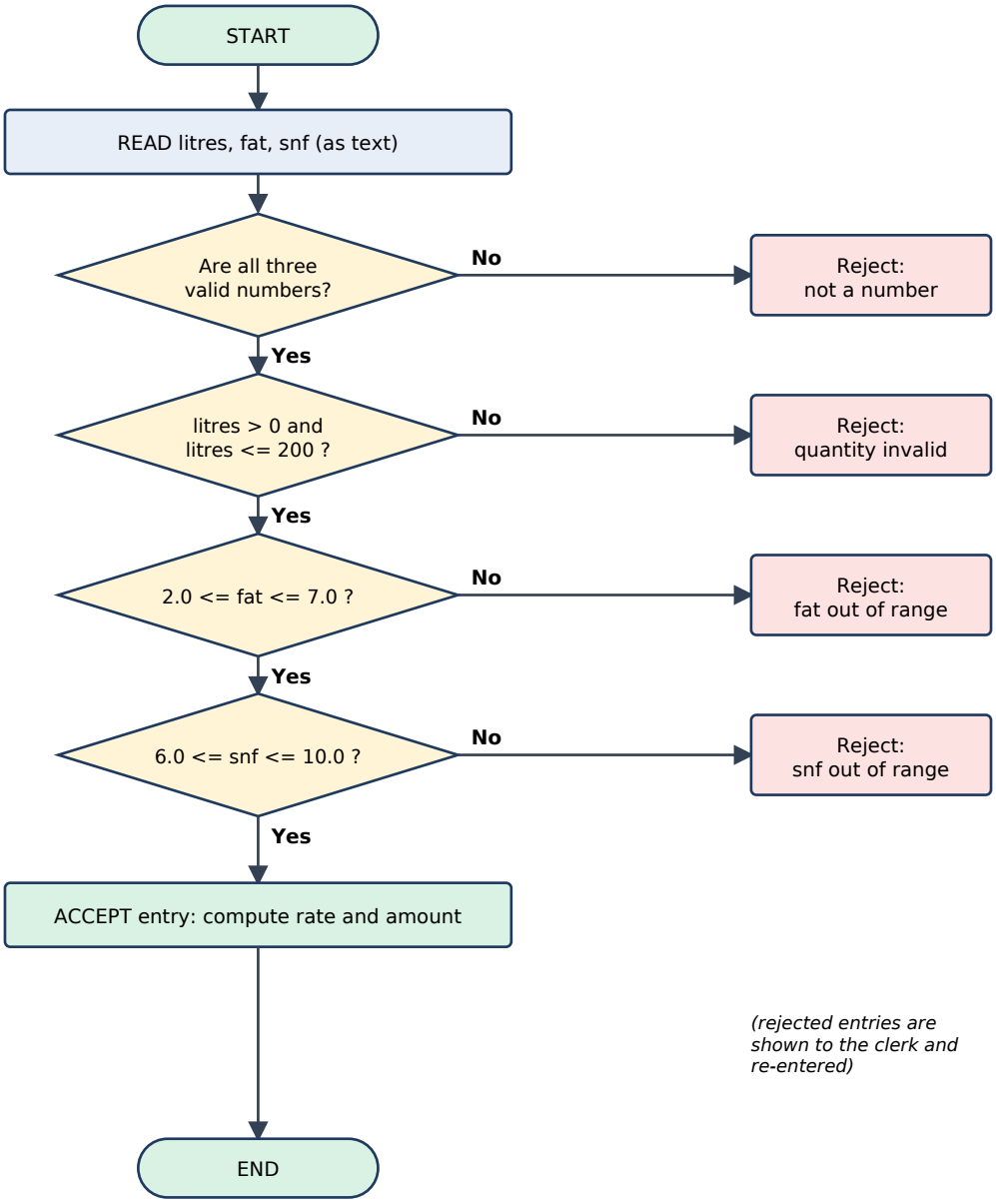
Pseudocode

```
ALGORITHM TenDayPayment
INPUT   entries : list of (litres, fat, snf), one item per delivery in the ten days
OUTPUT  total   : amount payable to the farmer in rupees (2 decimals)
CONSTANT BASE_RATE = 30.00, FAT_STEP_VALUE = 0.40, SNF_STEP_VALUE = 0.30
CONSTANT STD_FAT = 3.5, STD_SNF = 8.5, STEP = 0.1, MIN_FAT = 2.0, MAX_FAT = 7.0

BEGIN
 1   SET total TO 0.00
 2   FOR EACH (litres, fat, snf) IN entries DO
 3       IF litres <= 0 OR fat < MIN_FAT OR fat > MAX_FAT THEN
 4           DISPLAY "Invalid entry"
 5           CONTINUE with the next entry
 6       END IF
 7       SET fatSteps TO ROUND((fat - STD_FAT) / STEP)
 8       SET snfSteps TO ROUND((snf - STD_SNF) / STEP)
 9       SET rate     TO ROUND(BASE_RATE + fatSteps * FAT_STEP_VALUE + snfSteps * SNF_STEP_VALUE, 2)
10       SET amount   TO ROUND(litres * rate, 2)
11       SET total    TO total + amount
12   END FOR
13   RETURN total
END
```

Note: the keywords (BEGIN/END, SET … TO, FOR EACH, IF … END IF, RETURN) and the numbered lines follow the usual structured-pseudocode conventions; if the SIMATS standard sheet prescribes different keywords or layout, only the wording of each line needs to change, not the logic.

3. Flowchart – validating one entry



4. Hand trace of the pseudocode for three entries

Entry	litres	fat	snf	fatSteps (l.7)	snfSteps (l.8)	rate (l.9)	amount (l.10)	total (l.11)
1	10	3.5	8.5	0	0	30.00	300.00	300.00
2	8.5	4.2	8.7	7	2	33.40	283.90	583.90
3	12	3.1	8.3	−4	−2	27.80	333.60	917.50

Line 9 for entry 3: 30.00 + (−4)(0.40) + (−2)(0.30) = 30.00 − 1.60 − 0.60 = 27.80; line 10: 12 × 27.80 = 333.60. The program output below agrees with the trace (the final total 917.50 equals ten_day_total in Task 6).

Program output (trace.py)

```
  i litres  fat  snf fat steps snf steps   rate   amount    total
  1     10  3.5  8.5         0         0  30.00   300.00   300.00
  2    8.5  4.2  8.7         7         2  33.40   283.90   583.90
  3     12  3.1  8.3        -4        -2  27.80   333.60   917.50
```

Alternative Concept and Complexity Analysis

Solution 1 – store every entry's amount in a list, add at the end: `amounts.append(payout(...))` then `sum(amounts)`. **Solution 2 – running total while reading** (the pseudocode above).

Measured with tracemalloc (two_ways.py); times include tracemalloc overhead

```
 n=   300 | list_then_add      3.9 ms  peak     34.1 KB  total 135524.20 | running_total      3.7 ms  peak      0.7 KB
 n=  6000 | list_then_add     75.9 ms  peak    661.6 KB  total 2754827.26 | running_total     72.6 ms  peak      0.7 KB
 n=150000 | list_then_add   2097.8 ms  peak  16488.0 KB  total 68772194.07 | running_total   1897.3 ms  peak      0.7 KB
```

	List then add	Running total
Time for n entries	O(n): one payout per entry + one pass to add	O(n): one payout per entry; addition is done during the same pass
Extra space	O(n): 16.5 MB peak at n = 150,000	O(1): about 0.7 KB at every n
Keeps individual amounts?	yes	no

Both are O(n) in time (the list version needs one extra pass to add the amounts, a small constant); the space differs: O(n) versus O(1). At district scale, running totals keep memory flat. **Storing the entries is still necessary** when the amounts are needed again: printing a per-entry statement for the farmer, sorting or finding the highest/lowest delivery, computing an average or median, auditing a payment, or correcting one wrongly entered value afterwards.

Evidence

Pseudocode, flowchart and hand trace that match the final program (Task 9 uses the same rules and constants).